

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	REST vs. RESTful: Diseño e Implementación de Servicios Distribuidos				
NÚMERO DE LA PRÁCTICA:	06	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	25/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l6
REPLICACIÓN DE EJERCICIOS RESUELTOS Ejercicio Resuelto: API RESTful de Gestión de Productos Siguiendo las indicaciones del docente, se implementó una API RESTful básica utilizando Python y el framework Flask. El backend gestiona un recurso de "productos" permitiendo operaciones GET, POST y DELETE.
<pre> from flask import Flask, jsonify, request app = Flask(__name__) productos = [{"id": 1, "nombre": "Laptop"}, {"id": 2, "nombre": "Mouse"}] @app.route("/productos", methods=["GET"]) def obtener(): return jsonify(productos) @app.route("/productos", methods=["POST"]) def agregar(): data = request.json productos.append(data) return jsonify({"mensaje": "Producto agregado"}), 201 @app.route("/productos/<int:id>", methods=["DELETE"]) def eliminar(id): global productos productos = [p for p in productos if p["id"] != id] return jsonify({"mensaje": "Producto eliminado"}) if __name__ == "__main__": app.run(debug=True) </pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Para el consumo de la API, se desarrolló un cliente HTML minimalista que emplea la Fetch API de JavaScript para realizar peticiones asíncronas al servidor y renderizar la lista de productos dinámicamente.

```

async function listar() {
  let r = await fetch("http://127.0.0.1:5000/productos");
  let datos = await r.json();
  let lista = document.getElementById("lista");
  lista.innerHTML = "";
  datos.forEach((p) => {
    lista.innerHTML += `<li>${p.nombre}</li>`;
  });
}

```

A continuación se muestra la validación de los endpoints mediante comandos curl, donde se observa la creación y eliminación exitosa de recursos, así como la persistencia temporal en memoria.

```

> curl -X GET http://127.0.0.1:5000/productos
[
  {
    "id": 1,
    "nombre": "Laptop"
  },
  {
    "id": 2,
    "nombre": "Mouse"
  }
]
> curl -X POST -H 'Content-Type: application/json' -d '{"id":3, "nombre":"Teclado"}' http://127.0.0.1:5000/productos
{"mensaje": "Producto agregado"}
> curl -X DELETE http://127.0.0.1:5000/productos/2
{"mensaje": "Producto eliminado"}
> curl -X GET http://127.0.0.1:5000/productos
[
  {
    "id": 1,
    "nombre": "Laptop"
  },
  {
    "id": 3,
    "nombre": "Teclado"
  }
]
>

```

Figura 1: Validación de la API de Productos mediante CURL.

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Ejercicio 1: API RESTful Biblioteca (Java + Spring Boot)

Se implementó un sistema de gestión bibliotecaria profesional utilizando Spring Boot. El controlador de la API expone endpoints para la gestión integral de libros, soportando la subida de imágenes de portada y almacenamiento en una base de datos SQLite.

```

@RestController
@RequestMapping("/api/books")
@CrossOrigin(origins = "**")
public class BookApiController {

  private final BookRepository bookRepository;
  private final Path rootPath = Paths.get("uploads");

  @Autowired
  public BookApiController(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
    try {
      if (!Files.exists(rootPath)) {
        Files.createDirectories(rootPath);
      }
    } catch (IOException e) {
      throw new RuntimeException("Could not initialize folder for upload!", e);
    }
  }

  @GetMapping
  public List<Book> getAllBooks() {
    return bookRepository.findAll();
  }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```

@GetMapping("/{id}")
public ResponseEntity<Book> getBookById(@PathVariable Long id) {
    return bookRepository
        .findById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@PostMapping(consumes = { "multipart/form-data" })
public ResponseEntity<?> registerBook(
    @RequestParam("title") String title,
    @RequestParam("author") String author,
    @RequestParam("isbn") String isbn,
    @RequestParam(value = "description", required = false) String description,
    @RequestParam(value = "imageUrl", required = false) String imageUrl,
    @RequestParam(value = "image", required = false) MultipartFile image
) {
    if (title == null || title.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Title is required");
    }
    if (author == null || author.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Author is required");
    }
    if (isbn == null || isbn.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("ISBN is required");
    }

    // Check if ISBN already exists
    if (bookRepository.findByIsbn(isbn.trim()).isPresent()) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(
            "Book with this ISBN is already registered"
        );
    }

    Book book = new Book();
    book.setTitle(title.trim());
    book.setAuthor(author.trim());
    book.setIsbn(isbn.trim());
    book.setDescription(description != null ? description.trim() : null);

    if (image != null && !image.isEmpty()) {
        try {
            String originalFilename = image.getOriginalFilename();
            String cleanFilename =
                originalFilename != null
                ? originalFilename.replaceAll("[^a-zA-Z0-9.-]", "_")
                : "image";
            String fileName = UUID.randomUUID().toString() + "_" + cleanFilename;

            Files.copy(image.getInputStream(), this.rootPath.resolve(fileName));
            book.setImageUrl("/uploads/" + fileName);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                "Could not upload the image: " + e.getMessage()
            );
        }
    } else if (imageUrl != null && !imageUrl.trim().isEmpty()) {
        book.setImageUrl(imageUrl.trim());
    }

    Book savedBook = bookRepository.save(book);
    return ResponseEntity.status(HttpStatus.CREATED).body(savedBook);
}

@PutMapping(value =("/{id}", consumes = { "multipart/form-data" })
public ResponseEntity<?> updateBook(
    @PathVariable Long id,
    @RequestParam("title") String title,
    @RequestParam("author") String author,
    @RequestParam("isbn") String isbn,
    @RequestParam(value = "description", required = false) String description,

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

@RequestParam(value = "imageUrl", required = false) String imageUrl,
@RequestParam(value = "image", required = false) MultipartFile image
) {
    Optional<Book> bookOptional = bookRepository.findById(id);
    if (bookOptional.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    Book book = bookOptional.get();

    if (title == null || title.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Title is required");
    }
    if (author == null || author.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Author is required");
    }
    if (isbn == null || isbn.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("ISBN is required");
    }

    // Check if ISBN is taken by another book
    Optional<Book> existingIsbn = bookRepository.findByIsbn(isbn.trim());
    if (existingIsbn.isPresent() && !existingIsbn.get().getId().equals(id)) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(
            "Book with this ISBN is already registered"
        );
    }

    book.setTitle(title.trim());
    book.setAuthor(author.trim());
    book.setIsbn(isbn.trim());
    book.setDescription(description != null ? description.trim() : null);

    if (image != null && !image.isEmpty()) {
        // Case A: New file uploaded. Remove old disk image if there was one.
        deleteLocalImage(book.getImageUrl());
        try {
            String originalFilename = image.getOriginalFilename();
            String cleanFilename =
                originalFilename != null
                ? originalFilename.replaceAll("[^a-zA-Z0-9.-]", "_")
                : "image";
            String fileName = UUID.randomUUID().toString() + "_" + cleanFilename;

            Files.copy(image.getInputStream(), this.rootPath.resolve(fileName));
            book.setImageUrl("/uploads/" + fileName);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                "Could not upload the image: " + e.getMessage()
            );
        }
    }
    else if (imageUrl != null) {
        String trimmedUrl = imageUrl.trim();
        if (trimmedUrl.isEmpty()) {
            // User explicitly cleared the cover image
            deleteLocalImage(book.getImageUrl());
            book.setImageUrl(null);
        }
        else if (!trimmedUrl.equals(book.getImageUrl())) {
            // Case B: A new public URL is specified, remove old local image
            deleteLocalImage(book.getImageUrl());
            book.setImageUrl(trimmedUrl);
        }
    }

    Book updatedBook = bookRepository.save(book);
    return ResponseEntity.ok(updatedBook);
}

@DeleteMapping("/{id}")
public ResponseEntity<?> deleteBook(@PathVariable Long id) {
    Optional<Book> bookOptional = bookRepository.findById(id);
    if (bookOptional.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

    }

    Book book = bookOptional.get();
    deleteLocalImage(book.getImageUrl());
    bookRepository.delete(book);
    return ResponseEntity.ok().body("Book deleted successfully");
}

private void deleteLocalImage(String imageUrl) {
    if (imageUrl != null && imageUrl.startsWith("/uploads/")) {
        String filename = imageUrl.substring("/uploads/".length());
        try {
            Path filePath = this.rootPath.resolve(filename);
            Files.deleteIfExists(filePath);
        } catch (IOException e) {
            System.err.println(
                "Could not delete file " + filename + ": " + e.getMessage()
            );
        }
    }
}
}
}

```

El modelo de datos se definió utilizando JPA, asegurando que cada libro posea atributos obligatorios como título, autor e ISBN único, además de una descripción opcional y una URL para la imagen de portada.

```

package com.lab06.e1.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Column;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(length = 2000)
    private String description;

    private String imageUrl;
}

```

La validación de la API se realizó mediante pruebas de carga de datos, verificando que las restricciones de unicidad del ISBN y el manejo de archivos multipart funcionen correctamente según los principios RESTful.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```
> curl -s http://localhost:8080/api/books | jq '.' | head -n 30
[
  {
    "id": 1,
    "title": "1984",
    "author": "George Orwell",
    "isbn": "0451524934",
    "description": "A dystopian novel set in a totalitarian society ruled by Big Brother, exploring themes of surveillance, control, and truth.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0451524934-L.jpg"
  },
  {
    "id": 2,
    "title": "To Kill a Mockingbird",
    "author": "Harper Lee",
    "isbn": "0446310786",
    "description": "The story of young Scout Finch and her father Atticus, a lawyer defending a Black man falsely accused of rape in the segregated South.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0446310786-L.jpg"
  },
  {
    "id": 3,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "isbn": "0743273567",
    "description": "A portrait of the Jazz Age, following the mysterious millionaire Jay Gatsby and his tragic obsession with the beautiful Daisy Buchanan.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0743273567-L.jpg"
  },
  {
    "id": 4,
    "title": "Project Hail Mary",
    "author": "Andy Weir",
    "isbn": "0593236574",
    "description": ""
  }
]
```

Figura 2: Pruebas de la API de Biblioteca mediante CURL.

Se desarrolló una interfaz web moderna y responsiva para interactuar con la biblioteca de forma intuitiva, permitiendo el registro y edición de libros mediante formularios dinámicos.

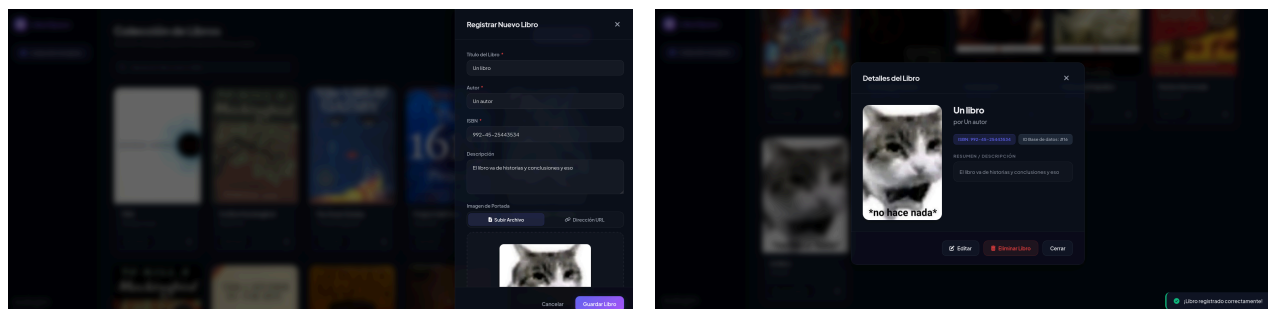


Figura 3: Interfaz gráfica de la biblioteca: Formulario de registro y vista de colección.

Ejercicio 2: API RESTful Estudiantes (Python)

Este ejercicio consistió en la creación de un sistema de registro estudiantil avanzado empleando Flask y SQLAlchemy. Se implementaron rutas para el registro, consulta, actualización y eliminación de perfiles académicos.

```
def register_routes(app):

    @app.route("/")
    def index():
        return send_from_directory(app.static_folder, "index.html")

    @app.route("/estudiantes", methods=["GET"])
    def listar():
        estudiantes = Estudiante.query.all()
        return jsonify([e.to_dict() for e in estudiantes])

    @app.route("/estudiantes", methods=["POST"])
    def agregar():
        data = request.json
        nuevo = Estudiante(
            matricula=data.get("matricula", ""),
            nombre=data.get("nombre", ""),
            carrera=data.get("carrera", ""),
            edad=data.get("edad", 0),
            email=data.get("email", ""),
            telefono=data.get("telefono", "")
        )
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

matriculado=bool(data.get("matriculado", True))
if isinstance(data.get("matriculado", True), bool)
else data.get("matriculado", "true") in ["true", True, 1],
semestres=data.get("semestres", 0),
estado=data.get("estado", "Activo"),
)
db.session.add(nuevo)
db.session.commit()
return jsonify({"ok": True})

@app.route("/estudiantes/<int:i>", methods=["PUT"])
def actualizar(i):
    estudiante = db.session.get(Estudiante, i)
    if estudiante:
        data = request.json
        if "matricula" in data:
            estudiante.matricula = data["matricula"]
        if "nombre" in data:
            estudiante.nombre = data["nombre"]
        if "carrera" in data:
            estudiante.carrera = data["carrera"]
        if "edad" in data:
            estudiante.edad = data["edad"]
        if "email" in data:
            estudiante.email = data["email"]
        if "telefono" in data:
            estudiante.telefono = data["telefono"]
        if "matriculado" in data:
            estudiante.matriculado = (
                data["matriculado"]
                if isinstance(data["matriculado"], bool)
                else data["matriculado"] in ["true", True, 1]
            )
        if "semestres" in data:
            estudiante.semestres = data["semestres"]
        if "estado" in data:
            estudiante.estado = data["estado"]

        db.session.commit()
        return jsonify({"actualizado": True})
    return jsonify({"error": "Estudiante no encontrado"}), 404

@app.route("/estudiantes/<int:i>", methods=["DELETE"])
def eliminar(i):
    estudiante = db.session.get(Estudiante, i)
    if estudiante:
        db.session.delete(estudiante)
        db.session.commit()
        return jsonify({"eliminado": True})
    return jsonify({"error": "Estudiante no encontrado"}), 404

```

La persistencia de datos se gestiona a través de un modelo ORM que mapea la entidad Estudiante a una tabla en SQLite, incluyendo campos para métricas académicas y estados de matriculación.

```

from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Estudiante(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    matricula = db.Column(db.String(50), nullable=False)
    nombre = db.Column(db.String(100), nullable=False)
    carrera = db.Column(db.String(100), nullable=False)
    edad = db.Column(db.Integer, nullable=False)
    email = db.Column(db.String(120), nullable=False)
    telefono = db.Column(db.String(30), nullable=False)
    matriculado = db.Column(db.Boolean, nullable=False, default=True)
    semestres = db.Column(db.Integer, nullable=False)
    estado = db.Column(db.String(40), nullable=False)

```

```
def to_dict(self):
    return {
        "id": self.id,
        "matricula": self.matricula,
        "nombre": self.nombre,
        "carrera": self.carrera,
        "edad": self.edad,
        "email": self.email,
        "telefono": self.telefono,
        "matriculado": self.matriculado,
        "semestres": self.semestres,
        "estado": self.estado
    }
```

A continuación se presenta la ejecución del cliente consumidor, el cual automatiza una serie de peticiones HTTP para validar el ciclo de vida completo de un recurso estudiantil en el servidor.

```
> curl -s http://localhost:5000/estudiantes | jq '.' | head -n 30
[
  {
    "carrera": "Ingeniería de Sistemas",
    "edad": 21,
    "email": "carlos.mendoza@campus.edu",
    "estado": "Activo",
    "id": 1,
    "matricula": "2024-IS-0142",
    "matriculado": true,
    "nombre": "Carlos Mendoza",
    "semestres": 5,
    "telefono": "+51 999 120 410"
  },
  {
    "carrera": "Medicina",
    "edad": 25,
    "email": "lucia.fernandez@campus.edu",
    "estado": "Activo",
    "id": 2,
    "matricula": "2023-MED-0898",
    "matriculado": true,
    "nombre": "Lucia Fernandez",
    "semestres": 8,
    "telefono": "+51 988 204 118"
  },
  {
    "carrera": "Arquitectura",
    "edad": 19,
    "email": "javier.solis@campus.edu",
    "estado": "En riesgo",
    "id": 3,
    "matricula": "2022-ARQ-0567",
    "matriculado": false,
    "nombre": "Javier Solis",
    "semestres": 2,
    "telefono": "+51 977 301 902"
  },
  {
    "carrera": "Diseño Gráfico",
    "edad": 22,
    "email": "mariana.rios@campus.edu",
    "estado": "Activo",
    "id": 4,
    "matricula": "2020-DG-0015",
    "matriculado": true,
    "nombre": "Mariana Rios",
    "semestres": 7,
    "telefono": "+51 966 221 665"
  }
]
```

Figura 4: Ejecución del cliente de pruebas para la API de Estudiantes.

Finalmente, se diseñó un panel administrativo (Dashboard) que ofrece visualizaciones estadísticas sobre el alumnado y herramientas de filtrado avanzado para la gestión de registros.

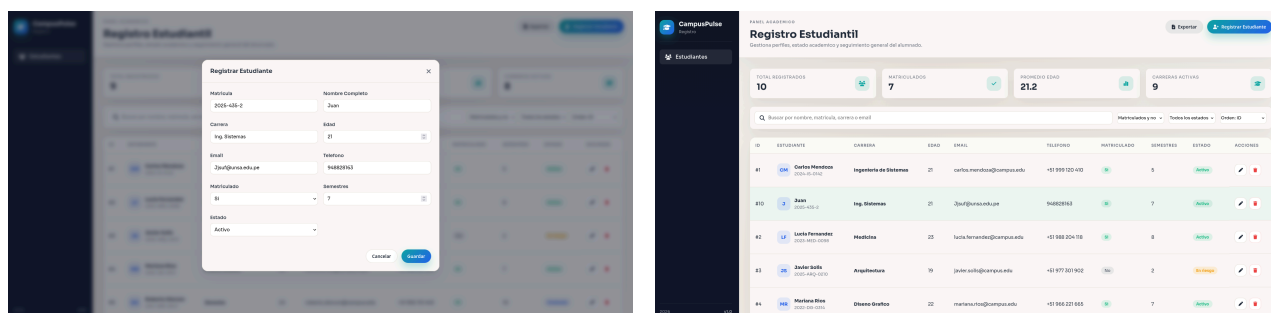


Figura 5: Panel de gestión estudiantil: Registro de alumnos y visualización del listado.

CUESTIONARIO

¿Por qué una API que utiliza HTTP no necesariamente puede considerarse RESTful?

Porque RESTful no se define simplemente por el uso del protocolo HTTP, sino por el cumplimiento estricto de las restricciones arquitectónicas de REST (Representational State Transfer). Muchas APIs utilizan HTTP solo como un túnel para RPC (Remote

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Procedure Call), donde los endpoints se nombran según acciones (/crearUsuario, /obtenerDatos) en lugar de recursos, y no respetan la semántica de los verbos HTTP ni la naturaleza sin estado o la interfaz uniforme (HATEOAS).

¿Qué consecuencias tendría diseñar endpoints orientados a acciones y no a recursos en sistemas distribuidos escalables?

Diseñar endpoints orientados a acciones (GET /deleteUser?id=5) rompe la uniformidad de la interfaz y dificulta la cacheabilidad. En sistemas distribuidos, esto genera una explosión de URIs difíciles de documentar y mantener. Además, impide que intermediarios (como proxies o balanceadores) optimicen las peticiones basándose en la semántica estándar de HTTP, limitando la escalabilidad horizontal y aumentando el acoplamiento entre cliente y servidor.

Compare RESTful frente a RPC y explique en qué escenarios empresariales RESTful podría ser una mala elección.

RESTful se centra en recursos y estado (escalable, desacoplado), mientras que RPC (como gRPC o RMI) se centra en procedimientos (eficiente, contrato fuerte). RESTful podría ser una mala elección en escenarios de baja latencia extrema (comunicación entre microservicios de alto rendimiento), donde el overhead de JSON y las cabeceras HTTP es excesivo. También es inadecuado para sistemas con flujos de datos complejos y multidireccionales que se benefician más de un contrato binario estricto y streaming.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. La arquitectura RESTful proporciona un estándar claro para la interoperabilidad en sistemas distribuidos, permitiendo que clientes heterogéneos consuman servicios mediante una interfaz uniforme.
2. La distinción entre REST (modelo teórico) y RESTful (implementación práctica) es fundamental para desarrollar APIs que realmente aprovechen las capacidades de optimización del protocolo HTTP.
3. El uso de frameworks modernos como Spring Boot y Flask simplifica enormemente la aplicación de principios RESTful, facilitando la gestión de estados y la serialización de datos.

RECOMENDACIONES

1. Se recomienda priorizar siempre el diseño orientado a recursos sobre el orientado a verbos para asegurar que la API sea intuitiva, documentable y compatible con estándares de industria.
2. Es fundamental utilizar correctamente los códigos de estado HTTP (200, 201, 404, 500, etc.) para que el cliente pueda manejar los errores y respuestas de forma semántica y predecible.
3. Para APIs destinadas a producción, se aconseja implementar mecanismos de autenticación (JWT) y limitación de tasa (rate limiting) para proteger los recursos expuestos.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- [4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
- [5] Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. Dissertation. University of California, Irvine.
- [6] Pautasso, O., Zimmermann, O., & Leymann, F. (2008). RESTful Web Services vs. "Big" Web Services: Making the Right Architectural Decision. WWW '08.